

EXERCISES

Redis Practical Exercises

Ten hands-on exercises · 60 marks · 2.5 hours

Module 07 · NoSQL & Redis

ICICI Bank Learning Academy

Enterprise Data Engineering with SQL, Redis & Kafka

Module 07 · NoSQL & Redis · Instructor-led · For classroom use

Before you start

Ten exercises, every one of them done at the keyboard. Four are `redis-cli` drills, five are Node.js programs you write, and one is an operations task where you deliberately break a Redis instance and then write the script that would have told you. Nothing here is answered by writing an essay: the deliverable is a transcript or a file that runs.

	Detail
Exercises	10 - four CLI, five programs, one operations
Total marks	60
Suggested time	2.5 hours, or take-home
You need	Redis 7.x · Node 18+ · <code>npm i redis</code> · a shell
Open book	Everything. The slides, the labs, the cookbook, the internet.
Hand in	The files named in each Deliverable line, plus your output

Section	Exercises	Marks	Drawn from
redis-cli drills	1 - 4	21	Decks 3 and 5, labs 7.2 - 7.6
Node.js programs	5 - 9	32	Deck 5, labs 7.9 - 7.10, the cookbook
Operations	10	7	Deck 4, lab 7.8

WATCH OUT Write the programs before you open `Node_Redis_Cookbook.pdf`. Several exercises are deliberately close to a recipe in it, and reading the answer first is the one way to get nothing out of this. Typing it wrong once is worth more than reading it right twice.

HOW IT IS MARKED Copy your terminal output into the answer. "It worked" is not a deliverable - most of the marks below are for showing the reply, the timing or the memory number you actually got.

REDIS-CLI DRILLS

redis-cli drills

Exercise 1 [5 marks]

Build and inspect the balance cache

Everything below is typed at the `redis-cli` prompt. Write the command *and* the reply you actually got - not the reply you expected.

You are given

```
$ redis-cli FLUSHDB
OK
$ redis-cli MSET icici:bal:9001 118450.00 icici:bal:9002 7320.55 \
                icici:bal:9003 341244.80
OK
```

What to do

1. Re-write 9003's balance so it expires five minutes from now, and prove the TTL is armed.
2. List every `icici:bal:*` key without blocking the server, and say in one line why `KEYS` is not acceptable in production.
3. Report the type and the internal encoding of `icici:bal:9003`, and explain the encoding in one line.
4. Create `icici:bal:9004` with value 5000 only if it does not exist. Run it twice, record both replies, then report its encoding and explain why it differs from 9003's.
5. Invalidate 9003 without blocking the server while the memory is freed.

DELIVERABLE A transcript covering all five steps.

Exercise 2 [6 marks]

Pick the structure, then actually build it

Four requirements from the ICICI platform. For each: name the type, build it with the data given, and run a verification command that *proves* it works. A type name on its own scores nothing.

What to do

1. **Recent transactions.** Account 9003 keeps only its 10 most recent transactions. Push `txn:1` to `txn:12` and prove exactly ten survive, with `txn:12` first.
2. **Branch leaderboard.** Load BR-MUM-014 250000, BR-DEL-002 410000, BR-BLR-031 185000, BR-CHN-009 322000. Add 90000 to BR-MUM-014 *without* reading its old score. Print the top three with scores.
3. **Duplicate payments.** REQ-88213 arrives twice. Record it so the *reply itself* identifies the duplicate, and give it a 24-hour life.
4. **Distinct logins.** Add 100,000 ids to a distinct-count structure and to a plain Set. Report MEMORY USAGE for both, and the error you measured.

DELIVERABLE A transcript per requirement. Task 4 also needs both memory numbers and the error, computed.

Exercise 3 [5 marks]

Re-model a JSON blob, and watch the encoding flip

The customer profile is one JSON string. Two servers update it at the same time and one update keeps disappearing. Re-model it, then look at what Redis does underneath.

You are given

```
$ redis-cli SET icici:cust:9003 \  
    '{"name":"A. Sharma","tier":"gold","logins":41}'  
OK
```

What to do

1. Store the same three fields as a Hash, plus branch, mobile and kyc. One command.
2. Record one extra login and read back only the tier - two commands, no client-side parsing.
3. Say in one line which concrete failure of the JSON version that removes.
4. Report OBJECT ENCODING and MEMORY USAGE for the Hash.
5. Make the encoding change. Show the command that did it, the new encoding and the new memory, and explain the jump in one line.

DELIVERABLE Transcript, plus the four numbers (two encodings, two memory readings).

Exercise 4 [5 marks]

Debug the session store

This script is in production. Sessions should die after 30 minutes of inactivity; instead Redis memory has grown every day for a month. Save it as `sessions.sh`, run it, and fix it.

You are given

```
#!/usr/bin/env bash
SID=8f3a
redis-cli SET icici:sess:$SID '{"cust":9003}' EX 1800

for i in 1 2 3; do
    # each incoming request
    redis-cli SET icici:sess:$SID "{\"cust\":9003,\"hits\":$i}"
    redis-cli TTL icici:sess:$SID
done

redis-cli KEYS 'icici:sess:*' | wc -l
```

What to do

1. Run it. Record the four TTL values and state the bug in one line.
2. Fix the expiry with a one-word addition. Re-run and record the TTLs.
3. The product owner now wants a *sliding* 30-minute window - every request resets the clock. Change your fix, then prove it: wait five seconds between two requests and show the TTL going back up.
4. Replace the KEYS line with something safe that returns the same count.
5. Add one line reporting how many keys Redis has expired since it started.

DELIVERABLE `sessions_fixed.sh` plus the before-and-after TTL transcripts.

NODE.JS PROGRAMS

Node.js programs

Exercise 5 [6 marks]

otp.js — OTP issue, verify and lock-out

Write the OTP service for the mobile app. An OTP is valid for 90 seconds, may be verified at most three times, and a fourth attempt locks the customer out for five minutes. It must be correct when the customer taps "verify" twice on two devices at the same instant.

You are given

```
$ cd node_cookbook && npm install
$ node otp.js # your file
```

What to do

1. Export `issue(mobile)` returning a 6-digit code, stored so it expires by itself after 90 seconds.
2. Export `verify(mobile, code)` returning one of OK, WRONG, EXPIRED, LOCKED.
3. Count attempts server-side. The counter must expire with the OTP, not outlive it, and must not be re-armed by a later attempt.
4. A correct code consumes the OTP - verifying the same code twice must not return OK twice.
5. Lock the mobile number for 300 seconds after the third wrong attempt.
6. A harness that prints: issue, one wrong, one right, a replay of the right one, then three wrongs to trigger the lock-out.

DELIVERABLE otp.js and its printed output.

Exercise 6 [7 marks]

cache.js — cache-aside, and its two failure modes

You are given a stub "core banking" module whose every read costs 50 ms. Put Redis in front of it, then reproduce and fix the two bugs every cache eventually has.

You are given

```
// core.js - given to you, do not change it
export const core = {
  rows: { 9001: 118450.00, 9002: 7320.55, 9003: 341244.80 },
  async selectBalance(acct) { await sleep(50); return this.rows[acct] ?? null; },
  async applyDebit(acct, amt) { await sleep(50); return (this.rows[acct] -= amt); },
};
```

What to do

1. Write `getBalance(acct)` returning `{ balance, source }` where `source` is `HIT` or `MISS`, with a 300-second TTL.
2. Measure it: print the cold (miss) time, the average warm (hit) time over 200 calls, and the speed-up.
3. **Bug 1.** Write `postDebit(acct, amount)`. Then show, with output, what a customer sees when the invalidation is missing - and fix it.
4. **Bug 2.** Populate 1,000 keys in one loop with an identical TTL, and explain what happens to the database when that TTL expires. Fix it in one line.
5. Make the API survive Redis being down: stop your Redis (or point `REDIS_URL` at a dead port) and show the program still returning balances.
6. Report the hit rate from `INFO stats` and state the threshold below which the cache is not earning its place.

DELIVERABLE `cache.js`, its output, and the two-line explanation of each bug.

Exercise 7 [6 marks]

lock.js — only one server runs the batch

End-of-day reconciliation runs at 23:30 on six app servers. Exactly one must run it. Twice would double-post every entry.

What to do

1. Write `acquire()` returning a token, or `null` if another server holds the lock. The lock must survive the holder crashing.
2. Write `release(token)` that deletes the lock *only* if this process still owns it. Explain in one line why two commands cannot do this safely.
3. Write `extend(token, ms)` for work that outlives the lease.
4. Prove it: run three workers concurrently and show exactly one ran the batch, with the other two reporting how long they must wait.
5. Prove the token check works: attempt a release with the wrong token and show it is refused, then release with the right one.
6. State in one line the failure this design still has on a replicated Redis.

DELIVERABLE `lock.js` and the output of the three-worker race.

Exercise 8 [6 marks]

queue.js — a job queue that does not lose work

"Email me my statement" queues a PDF job consumed by several workers. Build the queue, then kill a worker mid-job and prove nothing was lost.

What to do

1. Write a producer that queues 10 statement jobs.
2. Write a worker that claims a job, "generates the PDF" (sleep 30 ms) and acknowledges it. Claiming and acknowledging must be two separate steps.
3. Run two workers concurrently and have the second crash on `stmt - 4`, before acknowledging.
4. With the queue empty, show where `stmt - 4` actually is.
5. Write `requeueStale(worker)` that returns it to the queue, then run a third worker and show it completes.
6. State in one line why BRPOP alone cannot do this.

DELIVERABLE `queue.js` and the transcript of the crash and the recovery.

Exercise 9 [7 marks]

dashboard.js — four structures, one report

The regional head wants one screen, refreshed every ten seconds, from live data. Each figure needs a different structure - choosing correctly is most of the marks.

What to do

1. **Seed** 30 days of activity: 20 branches with volumes, 50,000 customer logins spread over the days, and per-day activity for 5,000 customers.
2. **Top 5 branches** this month, with scores, highest first.
3. **This branch's position** - given BR-CHN-009, print its rank and score without fetching the other nineteen.
4. **Unique customers** today, and across the last 7 days. The weekly figure must not double-count anyone who visited on several days.
5. **Customers active on all 7 days** - a retention number.
6. **Total memory** used by all the keys your dashboard reads, and a one-line justification that this design would still fit at ICICI scale.
7. Print it as a single formatted report.

DELIVERABLE dashboard.js and the printed report.

OPERATIONS

Operations

Exercise 10 [7 marks]

Break it, then write the health check

Do this on a scratch instance on port 6380, never on a shared one. You are going to run Redis out of memory on purpose, watch what it does, and then write the script an on-call engineer would run at 02:00.

You are given

```
$ redis-server --port 6380 --maxmemory 16mb \
               --maxmemory-policy noeviction --daemonize yes
$ redis-cli -p 6380 PING
PONG
```

What to do

1. Fill it until it breaks. Record the *exact* error text and say which config setting produced it.
2. Make writes succeed again with two CONFIG SET commands, and say what you must run so the change survives a restart.
3. Prove eviction is now happening, using a counter from INFO.
4. Report used_memory_human, maxmemory_human and mem_fragmentation_ratio, and say what a ratio of 1.6 would mean.
5. Say in one line why this fix would be wrong on the instance holding the job queue from exercise 8.
6. Write health.js: it prints version, uptime, clients, memory, hit rate, evicted and expired keys, fragmentation, last save status and the three slowest recent commands.
7. It must exit 0 healthy, 1 warning, 2 critical, so it can be wired into monitoring.

DELIVERABLE health.js, its output against both a healthy and a broken instance, and the transcript of tasks 1-5.

When you have finished

Mark your own work against the separate solutions document. For anything you got wrong, write down which deck, lab or cookbook recipe covers it - that list is the agenda for your revision, and it is worth more than the score.

Exercise	If it went badly, go back to
1 - 4	Deck 5 and labs 7.2 - 7.6 - re-run them, do not re-read them
5	Deck 5 · key expiry and atomic counters · cookbook 01, 08
6	Lab 7.9 and cookbook 02 - the four rules of cache-aside
7	Lab 7.10 and cookbook 14 - NX, PX and the token
8	Cookbook 05 and 12 - claim, acknowledge, recover
9	Deck 3 and cookbook 07, 09, 10 - choosing the structure
10	Deck 4 and lab 7.8 - maxmemory, eviction, persistence

SCORING Total: 60 marks. 42 or above (70%) means you are ready for the Module 07 assessment.